

工具类 (MidiNoteUtility / MidiMessageBuilder)

本页面介绍用于辅助 MIDI 1.0 收发及相关辅助功能的运行时工具类。

命名空间：`jp.kshoji.unity.midi.util`

位置：

- `Assets/MIDI/Scripts/Utilities/MusicTheory/MidiNoteUtility.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/MidiMessageBuilder.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/PitchBendUtility.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/MidiControlSmoothing.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/AutomationInterpolation.cs`
- `Assets/MIDI/Scripts/Utilities/Smf/` — `TempoMapExtractor`、`MeasureTimeUtility`、`TupletUtility`、`SwingUtility`、`MidiMetaMessageFactory`（详情见 [SMF 工具](#)）
- `Assets/MIDI/Scripts/Editor/UI/PianoKeyboardElement.cs`（`MidiKeyboardLogic`）

MidiNoteUtility

提供 MIDI 音符编号与音名之间的相互转换、八度计算、半音移调等功能的静态工具类。它是不依赖 Unity 或 MIDI 插件的纯 C# 实现。

主要常量

常量	值	说明
MiddleC	60	General MIDI 中的 C4
MinNote	0	最小音符编号
MaxNote	127	最大音符编号

API 一览

方法	说明	示例
ToNoteName(noteNumber, useSharps)	将音符编号转换为音名	60 → "C4"
TryParseNoteName(name, out noteNumber)	将音名转换为音符编号	"C#4" → 61
GetOctave(noteNumber)	获取八度编号	60 → 4
GetPitchClass(noteNumber)	获取音级 (0–11)	64 → 4 (E)
Transpose(noteNumber, semitones)	半音移调 (钳制到 0–127)	(60, 12) → 72
FromPitchClassAndOctave(pitchClass, octave)	根据音级与八度计算音符编号	(0, 4) → 60
FormatNoteWithNumber(noteNumber)	同时标注音名与编号	60 → "C4 (60)"

使用示例

```
using jp.kshoji.unity.midi.util;

// 音符编号 → 音名
Debug.Log(MidiNoteUtility.ToNoteName(60));           // "C4"
Debug.Log(MidiNoteUtility.ToNoteName(61, useSharps: false)); // "Db4"

// 音名 → 音符编号
if (MidiNoteUtility.TryParseNoteName("C#4", out var note))
    Debug.Log(note); // 61

// 移调
var transposed = MidiNoteUtility.Transpose(60, 7);    // G4 (67)
```

八度约定

本工具类使用 MIDI 标准的八度表示法。

```
octave = (noteNumber / 12) - 1
```

例：音符编号 60 → C4, 0 → C-1, 127 → G9

MidiMessageBuilder (MidiSend)

以 Fluent 接口封装 `MidiManager` 的发送 API，提供可链式调用的 MIDI 消息发送。其内部调用 `MidiManager.Instance.SendMidi*`。

入口点

方法	说明
<code>MidiSend.To(deviceId)</code>	获取面向指定设备的构建器
<code>MidiSend.ToFirstOutput()</code>	获取面向第一个输出设备的构建器

当 `deviceId` 为空时会抛出 `ArgumentException`；当不存在输出设备时会抛出 `InvalidOperationException`。

构建器方法

方法	说明
<code>Group(int)</code>	设置 MIDI 2.0 组 (0–15，默认 0)
<code>Channel(int)</code>	设置 MIDI 通道 (0–15，默认 0)
<code>NoteOn(note, velocity)</code>	发送 Note On (velocity 默认 127)
<code>NoteOff(note, velocity)</code>	发送 Note Off (velocity 默认 0)
<code>ControlChange(controller, value)</code>	发送 Control Change
<code>ProgramChange(program)</code>	发送 Program Change
<code>PitchWheel(amount)</code>	发送弯音 (14-bit, 0–16383, 中心 8192)
<code>PitchWheelNormalized(value)</code>	从归一化 float 发送弯音 (经由 <code>PitchBendUtility</code>)
<code>SystemExclusive(byte[])</code>	发送 SysEx
<code>AllNotesOff()</code> / <code>AllNotesOff(channel)</code>	向全部或指定通道发送 All Notes Off (CC 123)

每个方法都返回 `this`，因此可以链式调用。

使用示例

```
using jp.kshoji.unity.midi.util;

// 单次发送
MidiSend.To(deviceId).Channel(0).NoteOn(60, 100);

// 连续发送和弦
MidiSend.To(deviceId).Channel(0)
    .NoteOn(60, 127)
    .NoteOn(64, 127)
    .NoteOn(67, 127);

// 向第一个输出设备发送 CC
MidiSend.ToFirstOutput().Channel(0).ControlChange(1, 64);
```

与 MidiManager 的关系

`MidiSend` 是 `MidiManager` 的薄封装。它与现有的 `SendMidiNoteOn` 等方法行为一致。在 Unity 编辑器中，发送的消息会作为 OUT 显示在 [MIDI 监视器](#) 中。

关于详细的发送 API，另请参阅 [MIDI 1.0 运行时 \(MidiManager\)](#) 中的“发送 MIDI 1.0 消息”。

PitchBendUtility

14-bit 弯音辅助工具（中心 = 8192）。纯 C#，不依赖 Unity。

位置：`Assets/MIDI/Scripts/Utilities/Messaging/PitchBendUtility.cs`

成员	说明
<code>Min</code> / <code>Max</code> / <code>Center</code>	<code>0</code> / <code>16383</code> / <code>8192</code>
<code>Clamp(amount)</code>	钳制到 0–16383
<code>FromNormalized(normalized, zeroToOne)</code>	float → 14-bit（ <code>zeroToOne</code> ：0–1，或双极 $-1\dots+1$ ）
<code>ToNormalized(amount)</code>	14-bit → 0–1（中心 ≈ 0.5 ）
<code>Split</code> / <code>Combine</code>	LSB/MSB（7+7 bit）拆分/合并

尚未包含半音范围转换（参见 [索引](#) 中的未来功能）。

```
var amount = PitchBendUtility.FromNormalized(0.75f);
PitchBendUtility.Split(amount, out var lsb, out var msb);
MidiSend.To(deviceId).Channel(0).PitchWheel(amount);
```

MidiControlSmoothing

供 `MidiCcSmoother` 与序列器输出阶段共用的 EMA（指数移动平均）辅助工具。

位置：`Assets/MIDI/Scripts/Utilities/Messaging/MidiControlSmoothing.cs`

方法	说明
<code>ApplyEma(previous, raw, smoothFactor)</code>	EMA； <code>smoothFactor</code> 0.01–1.0（越小越平滑）
<code>ApplyEmaToMidi7(...)</code>	EMA 后四舍五入并钳制到 0–127
<code>ApplyEmaToMidi14(...)</code>	EMA 后四舍五入并钳制到 0–16383（弯音）

AutomationInterpolation

在指定 tick 处评估有序自动化断点（归一化 0-1 值）。

位置：`Assets/MIDI/Scripts/Utilities/Messaging/AutomationInterpolation.cs`

API	说明
<code>CurveKind.Linear</code> / <code>Smooth</code>	点之间的线性插值或 smoothstep
<code>Evaluate(points, tick, curve)</code>	tick 处的值；首点之前 / 末点之后取端点；空列表为 0

MidiKeyboardLogic

在编辑器的虚拟 MIDI 控制器键盘与运行时 UI 表面的 `MidiKeyboardElement` 之间共享的键盘布局工具类。

位置：`Assets/MIDI/Scripts/Editor/UI/PianoKeyboardElement.cs`（`MidiKeyboardLogic`）

API	说明
<code>IsBlackKey(note)</code>	判断是否为黑键
<code>CountWhiteKeysBefore(note)</code>	指定音符之前的白键数量
<code>CountWhiteKeys(start, end)</code>	范围内的白键数量
<code>DefaultStartNote</code> / <code>DefaultEndNote</code>	默认的 2 个八度范围 (C3–B4)

关于运行时 UI 的详情，请参阅 [流派套件](#)。

TempoMapExtractor

从 SMF（Sequence）按时间序列提取速度变化与拍号的静态工具类。可用于 SmfPlayer 的播放时间计算，以及作为生成谱面、时间轴的基础。

位置：Assets/MIDI/Scripts/Utilities/Smf/TempoMapExtractor.cs

API

方法	说明
Extract(Sequence)	从 Sequence 构建速度映射
Extract(byte[] smfData)	从 SMF 二进制提取
Extract(string filePath)	从 .mid 文件提取

TempoMap

成员	说明
TempoChanges	速度变化（Tick / 秒 / BPM）
TimeSignatures	拍号
TotalDurationSeconds	总播放时长
GetBpmAt(tick)	指定 Tick 处的 BPM
TickToSeconds(tick)	Tick → 秒
SecondsToTick(seconds)	秒 → Tick

关于详细的使用示例，请参阅 [SMF 工具](#)。同一文件夹还提供：

工具类	作用
MeasureTimeUtility	小节 tick 长度（TicksPerBar / TicksPerBeat）、小节 ↔ 时间
TupletUtility	有理连音符步进 tick（例如 3:2）；index↔tick 可逆
SwingUtility	仅播放时对偶数弱拍的 swing 延迟
MidiMetaMessageFactory	构建/读取 Tempo、Time Signature、Track Name 元事件

MidiScaleUtility

判断音符是否属于指定音阶的静态工具类。也会被 Visual Scripting 的 **Is Note In Scale** 节点使用。

MidiScaleType

值	说明
Major	大调音阶
NaturalMinor	自然小调音阶
HarmonicMinor	和声小调音阶
MajorPentatonic	大调五声音阶
MinorPentatonic	小调五声音阶
Blues	布鲁斯
Dorian	多利亚
Mixolydian	混合利底亚
Chromatic	半音阶

API

方法	说明
IsNoteInScale(noteNumber, rootNote, scaleType)	音符是否包含在音阶中
GetIntervals(scaleType)	音阶的半音音程数组

```
using jp.kshoji.unity.midi.util;

// D4(62) 包含在 C 大调音阶 (根音 C4=60) 中
bool inScale = MidiScaleUtility.IsNoteInScale(62, 60, MidiScaleType.Major); // true
```

ScaleUtility / ChordRecognition

ScaleUtility 扩展了 MidiScaleUtility，可批量判定多个音符的音阶归属。

方法	说明
AllNotesInScale(notes, rootNote, scaleType)	是否所有音符都在音阶内
AnyNoteOutsideScale(notes, rootNote, scaleType)	是否有任意音符在音阶外
GetIntervals(scaleType)	MidiScaleUtility.GetIntervals 的别名

ChordRecognition.Recognize(activeNotes) 根据按下的音符集合推断和弦名（C、Cm7、Cmaj7 等）。支持三和弦、七和弦的主要模式。

```
var notes = noteTracker.GetActiveNotes(0);
var chord = ChordRecognition.Recognize(notes);
var inScale = ScaleUtility.AllNotesInScale(notes, rootNote: 60, MidiScaleType.Major);
```

关于详细的实时检测，请参阅 [面向游戏玩法的组件](#) 中的 MidiChordDetector。

相关文档

- [MIDI 1.0 运行时 \(MidiManager\)](#)
- [SMF 工具 \(SmfPlayer / MidiRecorder\)](#)
- [编辑器工具 \(MIDI 监视器\)](#)
- [入门指南](#)